

C# / .NET Learning Roadmap

Topic Notes — Section 5: LINQ (Language Integrated Query)

Topics: 5.1 through 5.11

Date: 29 June 2026

Section 5 — LINQ (Language Integrated Query)

This section covers LINQ in full — all 11 roadmap topics across query syntax, method syntax, deferred execution, filtering, projection, ordering, grouping, joining, aggregation, set operations, quantifiers, element operators, and IQueryable vs IEnumerable.

5.1. Query Syntax vs Method Syntax | 5.10. Deferred vs Immediate Execution | 5.11. IQueryable vs IEnumerable

Builds on: 4.5 (IEnumerable — LINQ operates on anything implementing it), 3.1 (generics — LINQ methods are generic), 2.9 (interfaces — IQueryable extends IEnumerable), 1.8 (loops — LINQ replaces many manual loops).

What is LINQ and why does it exist?

Without LINQ, querying, filtering, transforming, and aggregating collections requires manual loops with accumulator variables — verbose and hard to read. LINQ (Language Integrated Query) is a set of extension methods on IEnumerable and IQueryable that provide a unified, composable, readable syntax for querying any data source.

```
// Without LINQ -- manual loop:
var result = new List<Order>();
foreach (var order in orders)
    if (order.Status == "Pending" && order.Amount > 100)
        result.Add(order);
result.Sort((a, b) => b.Amount.CompareTo(a.Amount));

// With LINQ -- one readable pipeline:
var top5 = orders
    .Where(o => o.Status == "Pending" && o.Amount > 100)
    .OrderByDescending(o => o.Amount)
    .Take(5)
    .ToList();
```

Two syntax styles — method syntax and query syntax

LINQ has two equivalent syntaxes. The compiler translates both to the same thing.

```
// Method syntax (fluent) -- the standard in modern .NET:
var result = orders
    .Where(o => o.Amount > 100)
    .Select(o => o.CustomerName)
    .Distinct()
    .OrderBy(name => name)
    .ToList();

// Query syntax -- SQL-like keywords, compiler translates to method syntax:
var result = (from o in orders
    where o.Amount > 100
    select o.CustomerName)
    .Distinct()
    .OrderBy(name => name)
```

```
.ToList();
```

Both produce identical IL. Method syntax is far more common in modern .NET — more compact, works for all LINQ methods (some like `Distinct`, `Take`, `Skip` have no query syntax equivalent), and IntelliSense works better with it. Know query syntax to read legacy code.

Method syntax	Query syntax
-----+-----	
<code>.Where(o => ...)</code>	<code> where o. ...</code>
<code>.Select(o => o.Name)</code>	<code> select o.Name</code>
<code>.OrderBy(o => o.Name)</code>	<code> orderby o.Name</code>
<code>.GroupBy(o => o.Status)</code>	<code> group o by o.Status</code>
<code>.Join(...)</code>	<code> join ... on ... equals ...</code>
No equivalent	<code> from, let, into</code>

5.10. Deferred vs immediate execution

LINQ queries do not execute when you write them. They execute when you iterate.

```
// This builds a query description -- nothing runs yet:
var query = orders
    .Where(o => o.Amount > 100)
    .OrderByDescending(o => o.Amount);
```

```
Console.WriteLine("Query built");    // prints FIRST
```

```
// Execution happens here -- when you iterate:
foreach (var order in query)
    Console.WriteLine(order.Id);
```

```
// Or when you call a materialising method:
var list = query.ToList();    // executes and stores result
var count = query.Count();    // executes and counts
var first = query.First();    // executes and gets first item
```

The query captures a reference to the source, not a snapshot. Every time you iterate, it re-evaluates:

```
var orders = new List<Order> { new Order { Id = 1, Amount = 200m } };

var query = orders.Where(o => o.Amount > 100);

orders.Add(new Order { Id = 2, Amount = 300m });    // added AFTER query defined

var result = query.ToList();
// result contains BOTH Id=1 and Id=2 -- deferred execution sees latest state
```

Materialising methods force immediate execution:

<code>.ToList()</code>	<code>// List<T></code>
<code>.ToArray()</code>	<code>// T[]</code>
<code>.ToDictionary()</code>	<code>// Dictionary<TKey, TValue></code>
<code>.ToHashSet()</code>	<code>// HashSet<T></code>
<code>.Count()</code>	<code>// int</code>
<code>.Sum()</code>	<code>// numeric</code>
<code>.Min()</code> / <code>.Max()</code>	<code>// smallest/largest</code>
<code>.Average()</code>	<code>// numeric average</code>
<code>.First()</code> / <code>.FirstOrDefault()</code>	

```
.Single() / .SingleOrDefault()
.Any() / .All()
.Contains()
```

5.11. IQueryable vs IEnumerable

This distinction is critical for EF Core:

```
// IQueryable<T> -- LINQ translates to SQL, runs on database:
var orders = await context.Orders
    .Where(o => o.UserId == userId)      // SQL: WHERE UserId = @userId
    .OrderBy(o => o.CreatedAt)           // SQL: ORDER BY CreatedAt
    .Take(20)                           // SQL: TOP 20
    .ToListAsync();                      // executes ONE SQL query

// IEnumerable<T> -- LINQ runs in memory (C# code):
var orders = context.Orders.ToList()    // loads ALL orders from DB first!
    .Where(o => o.UserId == userId)      // filters in C# -- wasteful
    .OrderBy(o => o.CreatedAt)
    .Take(20);
```

The rule: Keep your query as IQueryable for as long as possible. Only call .ToList() or .ToListAsync() at the very end after all filters are applied.

Key Takeaways

- LINQ = composable, readable data queries on any IEnumerable or IQueryable
- Method syntax is the standard — chain extension method calls with lambdas
- Deferred execution — LINQ builds a query description; execution happens on iteration or materialisation
- Materialising methods (ToList, Count, First, etc.) force immediate execution
- On IQueryable (EF Core), LINQ translates to SQL — keep queries as IQueryable and materialise last

Memory aid: LINQ is a conveyor belt with attachments. Each operator clips onto the belt and does its job as items pass by. The belt only starts moving when you press the materialise button (ToList, Count, First).

Debugging Tips

Symptom	Cause	Fix
LINQ query runs multiple times unexpectedly	Iterating an IEnumerable multiple times — deferred re-execution	Call .ToList() once and reuse the list
EF Core loads all rows then filters	.ToList() called before .Where() — query runs as IEnumerable	Move Where before ToListAsync() to keep it on IQueryable
InvalidOperationException: Sequence contains no elements	.First() called on empty sequence	Use .FirstOrDefault() and null-check the result
LINQ on EF Core throws could not be translated	Lambda contains C# code EF cannot translate to SQL	Move untranslatable logic to in-memory processing after ToListAsync()

5.2. Filtering (Where) and Projection (Select, SelectMany)

Builds on: 5.1 (deferred execution, IQueryable), 3.1 (generics — Select transforms T to TResult), 1.8 (loops — these replace the most common loop patterns).

Where — filtering

Where keeps items that satisfy a predicate, discards the rest. The LINQ equivalent of an if inside a foreach.

```
// Basic filter:
var pending = orders.Where(o => o.Status == "Pending");

// Compound condition:
var largePending = orders.Where(o => o.Status == "Pending" && o.Amount > 100);

// Where with index -- keep items at even positions:
var evenPositioned = orders.Where((order, index) => index % 2 == 0);

// Building conditional queries on IQueryable<T>:
IQueryable<Order> query = _context.Orders;

if (status is not null)
    query = query.Where(o => o.Status == status);

if (minAmount.HasValue)
    query = query.Where(o => o.Amount >= minAmount.Value);

var result = await query.ToListAsync();
// One SQL query with composite WHERE clause
```

Select — projection

Select transforms each item into something else — one output per input.

```
// Extract one property:
var names = orders.Select(o => o.CustomerName);

// Project to a DTO:
var dtos = orders.Select(o => new OrderDto
{
    Id            = o.Id,
    CustomerName  = o.CustomerName,
    Amount        = o.Amount
});

// Select with index:
var numbered = orders.Select((o, i) => new { Position = i + 1, Order = o });

// Select in EF Core -- runs as SQL SELECT column list:
var dtos = await _context.Orders
    .Where(o => o.UserId == userId)
    .Select(o => new OrderDto
    {
        Id            = o.Id,
```

```

        Customer = o.Customer.Name,    // EF handles the JOIN
        ItemCount = o.Items.Count      // COUNT subquery
    })
    .ToListAsync();

```

SelectMany — flattening nested collections

Select returns one output per input. SelectMany flattens: when your selector returns a collection, SelectMany unwraps it into one flat sequence.

```

var orders = new List<Order>
{
    new Order { Id = 1, Items = [new("Widget", 9.99m), new("Gadget", 24.99m)] },
    new Order { Id = 2, Items = [new("Widget", 9.99m)] },
};

// Select returns nested -- IEnumerable<List<OrderItem>>:
var nested = orders.Select(o => o.Items);
// [[Widget, Gadget], [Widget]]

// SelectMany flattens to IEnumerable<OrderItem>:
var allItems = orders.SelectMany(o => o.Items);
// [Widget, Gadget, Widget]

// Two-parameter overload -- keeping the parent:
var orderItems = orders.SelectMany(
    o => o.Items,
    (o, item) => new { OrderId = o.Id, ProductName = item.ProductName, Price = item.Price });

// SelectMany vs nested foreach:
var allTags = articles.SelectMany(a => a.Tags).ToList();

```

Key Takeaways

- Where(predicate) — keeps items satisfying the condition, discards the rest
- Select(selector) — transforms each item to a new shape; one output per input
- SelectMany(selector) — flattens nested collections; one-to-many unwrapping
- All three are deferred — nothing executes until materialisation
- Select in EF Core translates to SQL column list — only load columns you need

Memory aid: Where is a sieve — only the right stuff passes through. Select is a stamp — it reshapes everything. SelectMany is a bulldozer — it pushes nested piles into one flat pile.

Debugging Tips

Symptom	Cause	Fix
Select returns nested collections instead of flat	Used Select when SelectMany was needed	Replace Select(o => o.Items) with SelectMany(o => o.Items)
Where filter has no effect	Condition always evaluates to true, or filtering wrong property	Add a breakpoint in the lambda and inspect the values
EF Core Select throws could not be translated	Lambda contains C# logic with no SQL equivalent	Move untranslatable logic to after ToListAsync()

Symptom	Cause	Fix
Where on IQueryable after ToList is slow	ToList() called too early — filtering in memory	Move Where before ToListAsync() to filter in SQL

5.3. Ordering (OrderBy, ThenBy) | 5.4. Grouping (GroupBy) | 5.6. Joins (Join, GroupJoin)

Builds on: 5.2 (Where, Select), 4.6 (IComparable/IComparer — OrderBy uses the same comparison concepts), 5.1 (deferred execution), 4.2 (Dictionary — GroupBy produces a similar key-group structure).

5.3. OrderBy — sorting sequences

OrderBy produces a new ordered sequence without modifying the source. Unlike List.Sort() which sorts in place, LINQ ordering is non-destructive and deferred.

```
// Ascending:
var sorted = orders.OrderBy(o => o.Amount);

// Descending:
var sorted = orders.OrderByDescending(o => o.CreatedAt);

// Multi-key with ThenBy:
var sorted = orders
    .OrderBy(o => o.Status)           // primary
    .ThenByDescending(o => o.Amount) // secondary within same status
    .ThenBy(o => o.CustomerName);    // tertiary

// WRONG -- each OrderBy replaces the previous:
var sorted = orders.OrderBy(o => o.Status).OrderBy(o => o.Amount); // only Amount applies

// CORRECT -- use ThenBy for secondary sorts:
var sorted = orders.OrderBy(o => o.Status).ThenBy(o => o.Amount);

// Custom ordering -- status in business order:
var statusOrder = new Dictionary<string, int>
{
    ["Pending"] = 0, ["Processing"] = 1, ["Shipped"] = 2, ["Delivered"] = 3
};
var sorted = orders.OrderBy(o => statusOrder.GetValueOrDefault(o.Status, 99));

// EF Core -- translates to SQL ORDER BY:
var orders = await _context.Orders
    .OrderByDescending(o => o.CreatedAt)
    .ThenBy(o => o.Id)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

LINQ OrderBy is a stable sort — items with equal keys preserve their original relative order. EF Core requires OrderBy before Skip/Take.

5.4. GroupBy — grouping sequences

GroupBy partitions a sequence into groups where each group shares the same key. Each group is an `IGrouping` — a key plus a collection of items.

```
// Basic GroupBy:
var grouped = orders.GroupBy(o => o.Status);

foreach (var group in grouped)
{
    Console.WriteLine($"Status: {group.Key} -- {group.Count()} orders");
    foreach (var order in group)
        Console.WriteLine($"  Order {order.Id}: {order.Amount:C}");
}

// GroupBy with result selector -- cleanest form:
var summaries = orders.GroupBy(
    o => o.Status,
    (status, groupOrders) => new OrderStatusSummary
    {
        Status      = status,
        Count        = groupOrders.Count(),
        TotalAmount  = groupOrders.Sum(o => o.Amount),
        MaxAmount    = groupOrders.Max(o => o.Amount)
    })
    .OrderBy(s => s.Status)
    .ToList();

// Multi-key grouping -- composite anonymous type key:
var grouped = orders.GroupBy(o => new { o.Status, o.Currency });
foreach (var group in grouped)
    Console.WriteLine($"{group.Key.Status} / {group.Key.Currency}: {group.Count()} orders");

// GroupBy to Dictionary:
var ordersByStatus = orders
    .GroupBy(o => o.Status)
    .ToDictionary(g => g.Key, g => g.ToList());

// EF Core GroupBy -- translates to SQL GROUP BY:
var summary = await _context.Orders
    .Where(o => o.CreatedAt >= DateTime.UtcNow.AddDays(-30))
    .GroupBy(o => o.Status)
    .Select(g => new { Status = g.Key, Count = g.Count(), Total = g.Sum(o => o.Amount) })
    .ToListAsync();
// SQL: SELECT Status, COUNT(*), SUM(Amount) FROM Orders WHERE ... GROUP BY Status
```

5.6. Join and GroupJoin

Join combines items from two sequences based on matching keys — LINQ equivalent of SQL INNER JOIN.

```
// Method syntax:
var result = orders.Join(
    customers,
    o => o.CustomerId,           // outer key
    c => c.Id,                   // inner key
```

```

(o, c) => new                // result selector
{
    OrderId = o.Id,
    Customer = c.Name,
    Amount = o.Amount
});
// Unmatched items from either side are excluded (INNER JOIN behaviour)

// Query syntax -- cleaner for joins:
var result = from o in orders
              join c in customers on o.CustomerId equals c.Id
              select new { OrderId = o.Id, Customer = c.Name, Amount = o.Amount };

// GroupJoin -- LEFT OUTER JOIN behaviour (keeps all outer items):
var result = customers.GroupJoin(
    orders,
    c => c.Id,
    o => o.CustomerId,
    (customer, customerOrders) => new
    {
        Customer = customer.Name,
        OrderCount = customerOrders.Count(),
        TotalSpent = customerOrders.Sum(o => o.Amount)
    });
// Karthik with 0 orders IS included -- unlike INNER JOIN

// In EF Core -- prefer navigation properties over manual Join:
var orders = await _context.Orders
    .Include(o => o.Customer)
    .Select(o => new { o.Id, CustomerName = o.Customer.Name, o.Amount })
    .ToListAsync();

```

Key Takeaways

- Never chain `OrderBy` calls — each replaces the previous; use `ThenBy` for secondary sorts
- `OrderBy` is stable — equal keys preserve original relative order
- `GroupBy` partitions sequence by key; each group is `IGrouping` with a `.Key` property
- `GroupBy` with result selector is the cleanest form — projects groups directly to summary objects
- `Join` = INNER JOIN; `GroupJoin` = LEFT OUTER JOIN
- In EF Core, prefer navigation properties and `Include` over manual `Join`

Operator	SQL equivalent	Returns
-----+-----+-----		
<code>OrderBy(key)</code>	ORDER BY key ASC	<code>IOrderedEnumerable<T></code>
<code>OrderByDescending(k)</code>	ORDER BY key DESC	<code>IOrderedEnumerable<T></code>
<code>ThenBy(key)</code>	, key ASC	<code>IOrderedEnumerable<T></code>
<code>GroupBy(key)</code>	GROUP BY key	<code>IEnumerable<IGrouping<K, T>></code>
<code>Join(...)</code>	INNER JOIN	<code>IEnumerable<TResult></code>
<code>GroupJoin(...)</code>	LEFT OUTER JOIN	<code>IEnumerable<TResult></code>

Memory aid: `OrderBy` lines people up in a queue. `ThenBy` breaks ties within that queue.

`GroupBy` separates the queue into rooms by a common trait. `Join` merges two separate queues

by matching wristbands.

Debugging Tips

Symptom	Cause	Fix
Second sort key has no effect	Used OrderBy twice instead of ThenBy	Replace the second OrderBy with ThenBy
EF Core throws on Skip/Take without ordering	Database requires ORDER BY for pagination	Add OrderBy before Skip/Take
GroupBy result cannot be used directly	IGrouping is not a list -- must iterate or project	Use .Select(g => new { Key = g.Key, Items = g.ToList() }) or foreach
Join excludes items that should be included	INNER JOIN behaviour -- unmatched items dropped	Use GroupJoin for left outer join
EF Core GroupBy throws could not be translated	Grouping on a computed value EF cannot express in SQL	Group on a raw column; compute derived values in C# after materialising

5.5. Aggregation | 5.7. Quantifiers (Any, All, Contains) | 5.8. Element Operators

Builds on: 5.1 (deferred execution — all of these are immediate), 5.3 (GroupBy — aggregations appear inside GroupBy projections), 5.2 (Where — predicates work the same way here).

5.5. Aggregation — Sum, Count, Average, Min, Max, Aggregate

These are terminal aggregation operators — they consume a sequence and return a single scalar value immediately (no deferred execution).

```
decimal totalAmount = orders.Sum(o => o.Amount);
int      totalItems  = orders.Sum(o => o.ItemCount);

int count    = orders.Count();
int pending  = orders.Count(o => o.Status == "Pending");
long bigCount = orders.LongCount();

decimal minAmount = orders.Min(o => o.Amount);
decimal maxAmount = orders.Max(o => o.Amount);

// MinBy / MaxBy (.NET 6+) -- returns the OBJECT, not the value:
Order cheapest  = orders.MinBy(o => o.Amount);    // returns Order
Order priciest  = orders.MaxBy(o => o.Amount);    // returns Order
// vs Min/Max which return the VALUE:
decimal minVal  = orders.Min(o => o.Amount);      // returns decimal

double avgAmount = orders.Average(o => (double)o.Amount);
decimal avgDecimal = orders.Average(o => o.Amount);

// Min, Max, Average throw on empty sequences -- guard with Any():
decimal? min = orders.Any() ? orders.Min(o => o.Amount) : null;
// Or use nullable selector:
decimal? min = orders.Min(o => (decimal?)o.Amount); // null for empty
```

Aggregate — the general fold/reduce

Sum, Count, Min, Max, and Average are all special cases of Aggregate. It applies a function repeatedly across the sequence, accumulating a result.

```
var numbers = new[] { 1, 2, 3, 4, 5 };

// Sum manually with Aggregate:
int sum = numbers.Aggregate(0, (acc, n) => acc + n);
// Steps: 0+1=1, 1+2=3, 3+3=6, 6+4=10, 10+5=15

// String concatenation:
var words = new[] { "Hello", "world", "from", "LINQ" };
string sentence = words.Aggregate((acc, w) => $"{acc} {w}");
// "Hello world from LINQ"

// Building a dynamic WHERE clause string:
var conditions = new[] { "Status = 'Pending'", "Amount > 100", "UserId = 42" };
string where = conditions.Aggregate((acc, c) => $"{acc} AND {c}");
// "Status = 'Pending' AND Amount > 100 AND UserId = 42"

// Multiple aggregations in one pass:
var stats = orders.Aggregate(
    seed: (Count: 0, Total: 0m, Max: decimal.MinValue),
    accumulator: (acc, o) => (
        Count: acc.Count + 1,
        Total: acc.Total + o.Amount,
        Max: o.Amount > acc.Max ? o.Amount : acc.Max
    ));

// In EF Core -- one query for all aggregations:
var stats = await _context.Orders
    .GroupBy(_ => 1) // group everything into one group
    .Select(g => new
    {
        Count    = g.Count(),
        Total    = g.Sum(o => o.Amount),
        Min      = g.Min(o => o.Amount),
        Max      = g.Max(o => o.Amount),
        Average  = g.Average(o => o.Amount)
    })
    .FirstOrDefaultAsync();
```

5.7. Quantifiers — Any, All, Contains

Any() returns true if at least one element satisfies the condition. Stops as soon as it finds the first match.

```
// Any -- non-empty? or any match?
bool hasOrders = orders.Any();
bool hasPending = orders.Any(o => o.Status == "Pending");

// WRONG -- Count scans everything:
if (orders.Count() > 0) { ... } // O(n)
```

```

// CORRECT -- Any stops at first match:
if (orders.Any()) { ... }           // O(1) best case

// All -- every item must match:
bool allShipped = orders.All(o => o.Status == "Shipped");
bool allValid   = orders.All(o => o.Amount > 0);

// Empty sequence:
var empty = new List<Order>();
empty.Any(o => o.Amount > 0);    // false
empty.All(o => o.Amount > 0);    // true -- vacuously true!

// Logical relationship:
bool noPending = orders.All(o => o.Status != "Pending");
bool noPending2 = !orders.Any(o => o.Status == "Pending"); // equivalent

// EF Core -- Any translates to EXISTS:
bool has = await _context.Orders.AnyAsync(o => o.Status == "Pending");
// SQL: SELECT CASE WHEN EXISTS (SELECT 1 FROM Orders WHERE Status='Pending') THEN 1 ELSE 0 END

// EF Core -- All translates to NOT EXISTS:
bool all = await _context.Orders.AllAsync(o => o.Status == "Delivered");

```

Contains — checking membership:

```

// On a sequence -- checks if item is present:
var numbers = new[] { 1, 2, 3, 4, 5 };
bool has3 = numbers.Contains(3); // true
bool has9 = numbers.Contains(9); // false

// Contains with local collection in EF Core Where -- translates to SQL IN:
var allowedStatuses = new[] { "Pending", "Processing", "Shipped" };
var activeOrders = await _context.Orders
    .Where(o => allowedStatuses.Contains(o.Status))
    .ToListAsync();
// SQL: WHERE Status IN ('Pending', 'Processing', 'Shipped')

// ID lookup:
var ids = new[] { 1, 2, 3, 4, 5 };
var orders = await _context.Orders
    .Where(o => ids.Contains(o.Id))
    .ToListAsync();
// SQL: WHERE Id IN (1, 2, 3, 4, 5)

```

5.8. Element Operators — First, Single, ElementAt

```

// First -- returns first item, throws if empty:
Order first = orders.First();
Order firstPending = orders.First(o => o.Status == "Pending");

// FirstOrDefault -- returns first or null if empty:
Order? first = orders.FirstOrDefault();
Order? firstPending = orders.FirstOrDefault(o => o.Status == "Pending");

// Last / LastOrDefault:
Order last = orders.Last();

```

```

Order? maybe = orders.LastOrDefault(o => o.Status == "Pending");

// Single -- exactly ONE item must match, throws if 0 or >1:
Order unique = orders.Single(o => o.Id == 42);

// SingleOrDefault -- 0 or 1 match, throws if >1:
Order? unique = orders.SingleOrDefault(o => o.Id == 42);

// ElementAt -- item at specific index:
Order third = orders.ElementAt(2);
Order? safe = orders.ElementAtOrDefault(100); // null if out of range

// EF Core:
// Single/SingleOrDefault generates SELECT TOP(2) -- EF verifies uniqueness
// First/FirstOrDefault generates SELECT TOP(1)
// Prefer FirstOrDefaultAsync for most lookups

```

Operator	When sequence empty	When >1 match	Use when
First()	Throws	Returns first	Expect at least one
FirstOrDefault()	Returns null	Returns first	Item may not exist
Single()	Throws	Throws	Enforce exactly one
SingleOrDefault()	Returns null	Throws	Zero or one, no duplicates
Last()	Throws	Returns last	Want last item
ElementAt(i)	Throws	N/A	Need specific position

Key Takeaways

- Any() is always preferred over Count() > 0 for existence checks — short-circuits early
- All() returns true for empty sequences (vacuously true) -- guard with Any() first
- Contains() on a local collection inside EF Core Where translates to SQL IN (...)
- Min/Max return the VALUE; MinBy/MaxBy (.NET 6+) return the OBJECT
- Min, Max, Average throw on empty sequences -- guard with Any() or use nullable selectors
- GroupBy(_ => 1) gets all aggregations in one SQL query
- Prefer FirstOrDefaultAsync over SingleOrDefaultAsync unless uniqueness must be enforced

Operator	Returns	Empty seq	Short-circuits
Any()	bool	false	Yes
Any(predicate)	bool	false	Yes
All(predicate)	bool	true	Yes
Count()	int	0	No
Sum(selector)	numeric	0	No
Min(selector)	value type	Throws	No
MinBy(key)	T?	null	No
Average(sel)	double/decimal	Throws	No

.NET-specific rules:

- Async versions: AnyAsync, AllAsync, CountAsync, SumAsync, MinAsync, MaxAsync, AverageAsync
- MinBy/MaxBy available from .NET 6+
- On IQueryable: Any -> EXISTS, Count -> COUNT(*), Sum -> SUM(column)

Memory aid: Any is the optimist — stops the moment it sees one good thing. All is the pessimist — keeps looking for something wrong. Count is the accountant — tallies everything.

Debugging Tips

Symptom	Cause	Fix
InvalidOperationException: Sequence contains no elements	Min, Max, or Average called on empty sequence	Guard with Any() check, or use nullable selector: <code>Min(o => (decimal?)o.Amount)</code>
Any() slower than expected	Called on IQueryable without index on filtered column	Add a database index on the filtered column
All() returns true unexpectedly	Called on empty sequence -- vacuously true	Check Any() first before relying on All() result
MinBy/MaxBy not available	Targeting framework older than .NET 6	Upgrade to .NET 6+, or use <code>OrderBy().FirstOrDefault()</code> as workaround
Multiple async aggregation calls are slow	Running sequential awaits instead of concurrent	Use <code>Task.WhenAll</code> to run concurrent aggregation queries

5.9. Set Operations (Distinct, Union, Intersect, Except) | Other Operators

Builds on: 5.1 (deferred execution), 5.2 (Where, Select — these compose with them), 4.3 (HashSet — set operations use the same concepts).

Distinct and DistinctBy — removing duplicates

```
var numbers = new[] { 1, 2, 2, 3, 3, 3, 4 };
var unique = numbers.Distinct();    // [1, 2, 3, 4]

var statuses = orders.Select(o => o.Status).Distinct();
// ["Pending", "Shipped", "Cancelled"] -- no duplicates

// DistinctBy(.NET 6+) -- deduplicate by a key (more practical):
var onePerCustomer = orders.DistinctBy(o => o.CustomerId);

var unique = orders.DistinctBy(o => new { o.Status, o.Currency });
```

Union, Intersect, Except — set operations

```
var setA = new[] { 1, 2, 3, 4, 5 };
var setB = new[] { 3, 4, 5, 6, 7 };

setA.Union(setB)      // [1, 2, 3, 4, 5, 6, 7] -- all, no duplicates
setA.Intersect(setB)  // [3, 4, 5]             -- common items
setA.Except(setB)     // [1, 2]               -- in A but not B

// ...By variants (.NET 6+) -- key-based:
var commonOrders = freshOrders.IntersectBy(
    cachedOrders.Select(o => o.Id), o => o.Id);
```

```
var merged = newOrders.UnionBy(existingOrders, o => o.Id);
```

Aggregate, Zip, Chunk, Take, Skip, Concat, Range, Repeat

```
// Aggregate -- general fold/reduce (covered in 5.5 above)

// Take / Skip -- partitioning:
var numbers = new[] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
numbers.Take(3)           // [1, 2, 3]
numbers.Skip(3)           // [4, 5, 6, 7, 8, 9, 10]
numbers.TakeLast(3)       // [8, 9, 10]
numbers.SkipLast(3)       // [1, 2, 3, 4, 5, 6, 7]
numbers.TakeWhile(n => n < 5) // [1, 2, 3, 4]
numbers.SkipWhile(n => n < 5) // [5, 6, 7, 8, 9, 10]

// Zip -- combine two sequences element-by-element:
var names = new[] { "Sathish", "Priya", "Karthik" };
var scores = new[] { 95, 87, 92 };

var paired = names.Zip(scores, (name, score) => new { Name = name, Score = score });
// (Sathish, 95), (Priya, 87), (Karthik, 92)

// Without selector (.NET 6+) -- returns tuples:
var pairs = names.Zip(scores); // (Sathish, 95), ...
foreach (var (name, score) in pairs)
    Console.WriteLine($"{name}: {score}");

// Chunk (.NET 6+) -- split into batches:
var orderIds = Enumerable.Range(1, 250);
foreach (var batch in orderIds.Chunk(50))
    await _repo.ProcessBatchAsync(batch); // 5 batches of 50

// Concat -- join two sequences, preserving all including duplicates:
first.Concat(second) // [1,2,3,4,5,6] -- keeps duplicates
// vs Union which deduplicates

// Append / Prepend -- add single items:
items.Prepend(1).Append(5) // [1, 2, 3, 4, 5]

// Enumerable.Range and Repeat -- generate sequences:
Enumerable.Range(1, 5) // [1, 2, 3, 4, 5]
Enumerable.Repeat("pending", 3) // ["pending", "pending", "pending"]

// Generate test data:
var orders = Enumerable.Range(1, 100)
    .Select(i => new Order { Id = i, Amount = i * 10m })
    .ToList();
```

Key Takeaways

- `Distinct()` / `DistinctBy(key)` — remove duplicates; `DistinctBy` is more practical
- `Union` / `Intersect` / `Except` — set operations; `...By` variants for key-based operations
- `Concat` joins two sequences preserving all items including duplicates; `Union` deduplicates

- Zip pairs elements from two sequences by position; stops at shorter sequence
- Chunk(n) splits into batches of n; essential for bulk processing (.NET 6+)
- Enumerable.Range / Repeat generate sequences without a source

Operator	Purpose	Deferred?
Distinct/DistinctBy	Remove duplicates	Yes
Union/Intersect/Except	Set operations	Yes
Take / Skip	Partition sequence	Yes
TakeWhile/SkipWhile	Condition partition	Yes
Zip	Pair two sequences	Yes
Chunk	Split into batches	Yes
Concat	Concatenate	Yes
Append / Prepend	Add single items	Yes
Enumerable.Range	Generate integers	Yes
Enumerable.Repeat	Repeat a value	Yes

.NET-specific rules:

- DistinctBy, UnionBy, IntersectBy, ExceptBy, Chunk, three-way Zip require .NET 6+
- Aggregate without a seed uses first element as seed -- throws on empty sequence
- Except and Intersect use GetHashCode/Equals -- custom types need correct equality

Memory aid: Aggregate is a snowball rolling downhill — it accumulates as it goes. Zip is a zipper — it interleaves two sides into one. Chunk is a chocolate bar — scored into equal pieces.

Debugging Tips

Symptom	Cause	Fix
Distinct not removing duplicates on custom types	Custom type missing Equals/GetHashCode override	Use DistinctBy(key) or implement equality correctly
Zip result shorter than expected	Stops at the shorter sequence -- silent truncation	Ensure both sequences have same length, or handle explicitly
Chunk not available	Target framework older than .NET 6	Upgrade to .NET 6+ or write a custom Batch extension method
Except/Intersect not matching expected items	Default equality is reference-based for classes	Use ExceptBy/IntersectBy with a key selector, or implement IEquatable
Aggregate throws on empty sequence	No seed provided -- first element used as seed, but sequence is empty	Always provide a seed for sequences that might be empty

Section 5 complete. All 11 LINQ topics (5.1 through 5.11) from the roadmap are covered: query syntax, method syntax, deferred execution, IQueryable vs IEnumerable, Where, Select, SelectMany, OrderBy, ThenBy, GroupBy, Join, GroupJoin, Any, All, Contains, Count, Sum, Min, Max, Average, Aggregate, First, FirstOrDefault, Single, ElementAt, Distinct, Union, Intersect, Except, Zip, Chunk, and EF Core best practices. Say next to begin Section 6: Delegates, Events, and Functional Constructs.